

Scalable Taxi Trip Pattern Mining Using K-Means and MiniBatch K-Means

A Mini-Project Report

Submitted for the subject
Data Warehousing and Mining (DWM)

Department of Computer Engineering
Pune Institute of Computer Technology (PICT), Pune

Submitted By
Vighnesh Vijay Jadhav — Roll Number 31235
Shreesh Jugade — Roll Number 31238

Academic Year 2026–2027

Project application: Taxi Pattern Lab

Certificate

This page is provided for institutional certification after evaluation and signature. It does not represent an approval already granted.

This is to certify that the mini-project entitled “Scalable Taxi Trip Pattern Mining Using K-Means and MiniBatch K-Means” has been submitted by Vighnesh Vijay Jadhav (Roll Number 31235) and Shreesh Jugade (Roll Number 31238), Department of Computer Engineering, Pune Institute of Computer Technology (PICT), Pune, for the subject Data Warehousing and Mining during the academic year 2026–2027.

The work presented covers data preparation, clustering, empirical evaluation, visualization and prediction using saved clustering models. Certification of satisfactory completion is subject to the institution's evaluation process.

Faculty Guide: [Name to be entered]

Signature: _____ Date: _____

Subject Coordinator: [Name to be entered]

Signature: _____ Date: _____

Head of Department: [Name to be entered]

Signature: _____ Date: _____

Place: Pune

Acknowledgement

We acknowledge the Department of Computer Engineering at Pune Institute of Computer Technology, Pune, for the academic setting in which this Data Warehousing and Mining mini-project is submitted. The project provided an opportunity to connect clustering theory with a reproducible implementation and an evidence-based comparison under practical computing constraints.

We thank the New York City Taxi and Limousine Commission for making trip-record data publicly available. Access to a real, multi-million-record dataset enabled the study to examine data quality, computational scaling and interpretation beyond a small demonstration dataset.

We acknowledge the developers and research communities behind Python, Polars, NumPy, scikit-learn, UMAP, Streamlit and Plotly. Their software and published documentation form the foundation of the implementation. Relevant algorithms and external sources are cited in the references.

We also acknowledge the role of testing and version control in making the work inspectable. Explicit records of rejected inputs, resource decisions and experimental configurations were maintained so that the report can be evaluated against the accompanying repository.

Faculty-specific acknowledgements may be added after names and contributions have been confirmed. No faculty name, signature or institutional approval has been assumed in this report.

Vighnesh Vijay Jadhav and Shreesh Jugade

Abstract

Taxi-trip records provide a useful setting for studying scalable unsupervised learning because trip characteristics vary widely and raw records contain invalid or implausible values. This project develops Taxi Pattern Lab, a local analytics application that compares standard K-Means and MiniBatch K-Means using the official January 2025 New York City Yellow Taxi Parquet dataset. Of 3,475,226 source records, 3,241,580 satisfy the documented validation criteria. Six standardized features represent trip distance, duration, average speed, fare per mile and the sine and cosine of pickup hour.

A reproducible analysis of K values from 2 to 10 uses a 20,000-record sample, a fixed 2,000-record silhouette subset, cluster-size constraints and initialization stability. K=3 is selected as a defensible resolution for broad travel patterns. Although K=2 has the highest silhouette, K=3 provides a stable additional time-of-day distinction; larger values begin allocating centroids to very small groups influenced by extreme fare-per-mile values. Both final models are fitted on the same 100,000 records, while separate 2D and 3D UMAP reducers use 5,000 reference records.

Controlled local benchmarks complete paired experiments at 50,000, 100,000, 250,000, 500,000 and 1,000,000 records. At one million records, measured fitting times are 6.661 seconds for K-Means and 0.443 seconds for MiniBatch K-Means, with sampled adjusted Rand agreement of 0.9873. Ordinary and incremental MiniBatch runs also process the entire cleaned dataset. Full ordinary K-Means is explicitly skipped because its estimated memory requirement exceeds the local safety allowance. A free Google Colab fallback is implemented but was not executed.

The application presents cluster profiles, performance evidence, centroid-distance outliers and saved-model inference for unseen CSV or Parquet records. All accepted uploads receive cluster assignments without refitting. Twenty-five automated tests pass, and real holdout checks verify both algorithms and both UMAP transformations. The findings demonstrate a practical speed–quality trade-off while preserving explicit limits on sampling, memory and interpretation.

Keywords: clustering; taxi trips; K-Means; MiniBatch K-Means; scalability; UMAP; DWM.

Table of Contents

1	Introduction	8
2	Literature Review and Theoretical Background	9
2	Quality metrics and visualization theory	10
3	Dataset Description and Preprocessing	11
3	Preprocessing workflow and statistics	12
4	System Architecture and Methodology	13
4	Persistence and inference workflow	14
5	Algorithm Implementation	15
5	MiniBatch K-Means implementation	16
5	Cluster selection evidence	17
5	Selection rationale and sensitivity	18
6	Experimental Setup and Performance Evaluation	19
6	Runtime and memory results	20
6	Inertia silhouette and agreement	21
6	Full-data and batch-size experiments	22
7	Cluster Analysis and Knowledge Discovery	23
7	Interpretation of K-Means profiles	24
7	UMAP projections and interpretation limits	25
8	Anomaly Detection	26
9	Application Implementation	27
9	Comparison and anomaly pages	28
9	Prediction and methodology pages	29
10	Results and Discussion	30
11	Conclusion and Future Scope	31
	References	32
	Appendix A Installation and repository guide	33
	Appendix B Inputs outputs and validation	34

List of Figures

- Figure 1. Validation and feature engineering with measured population counts · 12
- Figure 2. Local analytical architecture and separate benchmark workflow · 13
- Figure 3. New-file prediction using immutable saved preprocessing and models · 14
- Figure 4. Elbow and silhouette analysis for K from 2 to 10 with K=3 marked · 18
- Figure 5. Fitting time and process-tree RSS at the five paired dataset sizes · 20
- Figure 6. Normalized inertia and sampled silhouette for identical paired inputs · 21
- Figure 7. Batch-size performance on one fixed 100000-record prefix · 22
- Figure 8. Training cluster distributions for both saved algorithms · 23
- Figure 9. Mean distance duration speed and fare intensity for K-Means groups · 24
- Figure 10. Saved 2D and 3D UMAP coordinates for the same 5000 reference trips · 25
- Figure 11. Reference-score distribution and all K-Means training outliers · 26
- Figure 12. Overview page with measured populations and categorical profiles · 27
- Figure 13. Cluster Discovery page displaying the saved reference projection · 27
- Figure 14. Actual Algorithm Comparison capture showing cluster-selection analysis · 28
- Figure 15. Anomaly Explorer screenshot placeholder; capture unavailable · 28
- Figure 16. Predict New Trips screenshot placeholder; capture unavailable · 29
- Figure 17. Methodology screenshot placeholder; capture unavailable · 29

List of Tables

Table 1. Dataset populations and exclusive cleaning outcomes ·	11
Table 2. Feature statistics across all accepted records ·	12
Table 3. Measured K-selection metrics on a fixed selection sample ·	17
Table 4. Recorded runtime and estimator configuration ·	19
Table 5. Paired fit times and peak RSS for K=3 ·	20
Table 6. Quality metrics and sampled partition agreement ·	21
Table 7. Completed MiniBatch batch-size study ·	22
Table 8. Saved-model cluster counts and mean trip characteristics ·	23
Table 9. Training outlier counts by model-specific cluster ·	26
Table 10. Repository components and reproducibility roles ·	33
Table 11. Actual examples from the disjoint 30-record holdout ·	34

1 Introduction

1.1 Background and problem statement

Data mining seeks useful structure in recorded observations. In an unsupervised task, labels describing the desired groups are not supplied. A clustering algorithm therefore constructs a partition from selected attributes and a similarity measure; interpreting that partition remains an analytical responsibility. Taxi-trip records contain varied travel distances, elapsed times, charges and pickup times within a common operational context [1].

The problem addressed is to identify interpretable taxi-trip patterns while comparing the computational cost and partition quality of K-Means and MiniBatch K-Means. The same cleaned observations, feature order, scaling parameters and frozen cluster count must be used in each paired experiment. A faster result is useful only when its quality and resource requirements are also reported.

1.2 Motivation and objectives

A modest laptop cannot treat ingestion, model fitting and interactive rendering as if they had identical memory requirements. The project consequently separates these stages. Its objectives are to quantify rejected records, engineer numerical features, justify K, persist both models, measure runtime and memory over nested data sizes, interpret the learned groups and demonstrate inference on new files without changing the partition.

The DWM contribution is a reproducible analytical preparation and mining workflow. The implementation does not claim to build a warehouse server, dimensional database or OLAP cube. Columnar Parquet storage, validation, provenance and reusable analytical artifacts supply the data-management foundation for mining.

1.3 Scope and expected outcomes

The empirical scope is one official monthly file. The local application contains six analytical pages and accepts compatible CSV and Parquet inputs. Outputs include comparison tables, cluster profiles, projections, anomaly flags and downloadable predictions. Geography, passenger intent, causal conclusions and confirmed fraud detection are outside the implemented scope.

The final model-training population is 100,000 accepted trips. Benchmark populations extend to all 3,241,580 accepted records. These distinct populations are stated throughout the report to prevent a full-dataset benchmark from being mistaken for the training size of the saved demonstration models.

2 Literature Review and Theoretical Background

2.1 Partition-based clustering

Partition methods assign each observation to one of K groups. K-Means represents each group by a centroid and minimizes squared Euclidean distances from observations to their assigned centers. Lloyd's iterative formulation alternates assignment and centroid recomputation [2]. The method is useful for numerical features but is sensitive to scales, initialization and extreme observations.

Sculley's MiniBatch approach replaces repeated full-data updates with small batches, reducing the work performed at each update [3]. It is relevant when rapid fitting is more important than reproducing exactly the conventional optimizer's partition. This project evaluates that trade-off on identical inputs rather than assuming the label vectors should match.

2.2 Distance and standardization

For vectors x and y containing d coordinates, Euclidean distance is defined by Equation (1). Squaring this distance gives the contribution used by the clustering objective. A feature measured in large numerical units can dominate this calculation even when it is not substantively more important.

$$d(x, y) = \sqrt{\sum_{j=1}^d (x_j - y_j)^2} \quad (1)$$

The saved StandardScaler applies Equation (2) using training means μ_j and population standard deviations σ_j . Each feature is centered and scaled before fitting. A constant feature requires a safe scale rather than division by zero. Standardization changes units; it does not remove skew, decorrelate variables or guarantee resistance to outliers.

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \quad (2)$$

Scikit-learn supplies tested estimator and transformation interfaces [4]. The implementation preserves the same fitted scaler for both algorithms and for uploads. Fitting a new scaler on each uploaded file would change the coordinate system and invalidate distances to saved centroids.

2 Quality metrics and visualization theory

2.3 Inertia and silhouette

Inertia J is the total squared distance to assigned centroids. Reporting J/n compares average distortion across sizes, although different samples can have different distributions. Increasing K generally reduces inertia, so minimum observed inertia alone cannot select a useful resolution.

$$J = \sum_{i=1}^n \min_{c \in \{1, \dots, K\}} \|z_i - \mu_c\|^2 \quad (3)$$

Let $a(i)$ be average distance from observation i to other points in its cluster and $b(i)$ the smallest average distance to another cluster. The silhouette in Equation (4) balances cohesion and separation [5]. Values near one indicate strong separation; negative values can indicate poor assignment. Sampled silhouette is not a full-population score.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (4)$$

2.4 Agreement and dimensionality reduction

Adjusted Rand Index (ARI) evaluates pairwise partition agreement while correcting for chance. Its conceptual form is (observed agreement – expected agreement)/(maximum agreement – expected agreement). It is invariant to permutations of cluster IDs [6]. ARI near one supports similar partitions, not equivalent semantic labels or accuracy against ground truth.

UMAP constructs a neighborhood representation and optimizes a lower-dimensional embedding [7]. It supports more than two dimensions and transformation of new records using a saved reducer [8]. Here it is solely a visualization stage. Clustering and quality metrics operate on six standardized features, never on UMAP coordinates.

2.5 Implications for this study

These methods motivate retaining a full-batch baseline, measuring the MiniBatch approximation and separating visualization from the objective. Centroid-distance anomaly flags reuse K-Means geometry; they cannot independently establish whether a trip is erroneous or fraudulent. The contribution is a controlled application under explicit resource limits, not a new clustering algorithm.

3 Dataset Description and Preprocessing

3.1 Source and provenance

The source is yellow_tripdata_2025-01.parquet from the official NYC TLC trip-record page [1]. The downloaded file contains 3,475,226 records and occupies 59,158,238 bytes. The SHA-256 is 9af277e4c0d3f9deb30644da822981e1e7df6af58313170fd3aa8a474485488a. The downloader retries failed transfers, verifies readable Parquet and replaces a temporary file after verification.

File membership defines the monthly source; pickup dates are not additionally clipped to January. Timestamps represent NYC local clock values. Timezone-aware uploads are rejected rather than silently converted. Provider records are observational and may contain errors.

Table 1. Dataset populations and exclusive cleaning outcomes.

Population / exclusion	Records
Raw January source	3,475,226
Accepted	3,241,580
Rejected total	233,646
Invalid charges	127,550
Invalid distance	64,474
Invalid duration	41,488
Implausible speed	134

3.2 Required attributes and features

Inputs require pickup timestamp, dropoff timestamp, trip_distance, fare_amount and total_amount. Total charge is validated but is not a clustering feature. Passenger count and geographic location IDs are excluded from the final matrix. Integer location identifiers do not represent meaningful Euclidean geographic distance without additional encoding.

Distance measures trip scale; duration measures elapsed time; average speed relates these quantities. Fare per mile describes charge intensity. Sine and cosine of pickup hour represent cyclical timing without placing midnight far from 23:00. These features are related, so standardization does not create six independent sources of information.

3 Preprocessing workflow and statistics

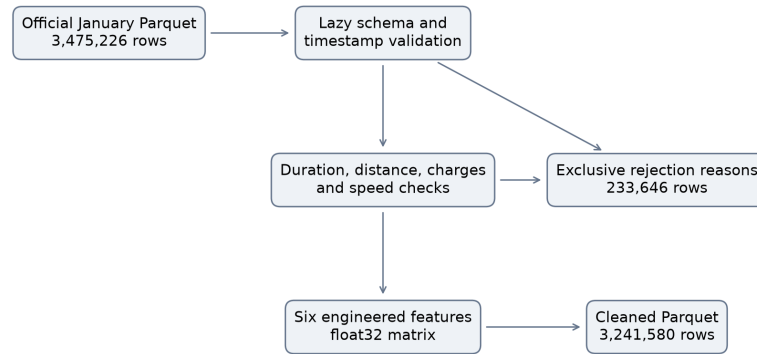


Figure 1. Validation and feature engineering with measured population counts.

3.3 Filtering and feature engineering

Validation generates a zero-based input `row_id` and assigns the first failing reason. Checks cover malformed timestamps, missing or non-finite values, duration outside 1–180 minutes, distance outside (0,150] miles, fare outside (0,1000], total outside (0,1500], speed above 100 mph and non-finite float32 features. These are project plausibility limits, not official fare regulations. Invalid values are rejected, not imputed.

Duration is elapsed seconds/60; speed is $60 \times \text{distance}/\text{duration}$; charge intensity is fare/distance. Zero distance and invalid duration cannot reach the model. Integer pickup hour h becomes $\sin(2\pi h/24)$ and $\cos(2\pi h/24)$. Extra columns are ignored.

Table 2. Feature statistics across all accepted records.

Statistic	Miles	Minutes	mph	USD/mile
mean	3.18	14.72	11.33	12.20
std	4.16	11.34	6.40	97.81
min	0.01	1.00	0.01	0.00
50%	1.70	11.72	9.62	7.17
max	148.30	179.97	99.40	11,776.00

The large fare-per-mile standard deviation and maximum show that accepted data retain an extreme tail. No winsorization, log transform or statistical outlier removal is applied. This affects the objective and helps explain tiny groups appearing at larger K .

4 System Architecture and Methodology

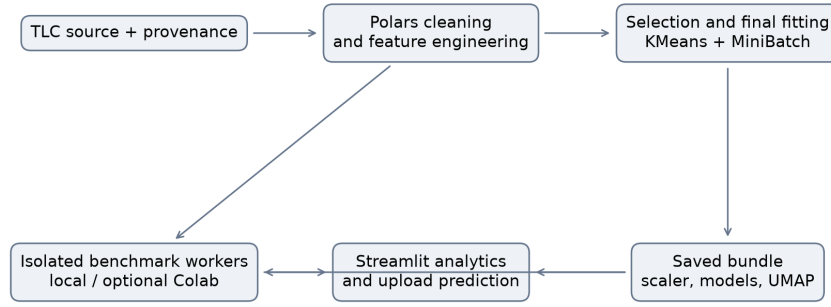


Figure 2. Local analytical architecture and separate benchmark workflow.

4.1 Separation of responsibilities

Polars lazily validates the source and writes cleaned Parquet. NumPy float32 arrays provide the interface to scikit-learn. Bounded selection establishes K before final fitting. The saved bundle contains the scaler, two estimators, validation rules, feature order, anomaly thresholds and separate UMAP reducers.

The benchmark coordinator prepares a standardized memory-mapped array in a short-lived process. Isolated workers fit each estimator and measure quality on identical data. CSV/JSON reports carry status, parameters, environment and a frozen-experiment fingerprint. Streamlit reads small reports and bounded reference data instead of repeatedly holding the source in memory.

4.2 Technology choices

Polars and Parquet support selective columnar processing; NumPy supplies compact matrices; scikit-learn supplies reproducible estimator APIs; joblib persists trusted local objects. UMAP supplies saved projections, Plotly interactive charts and Streamlit a local interface [9]. Pytest and Ruff provide automated checks and source consistency. These choices keep the project within one Python application.

The architecture supports local application use even when full conventional benchmarking cannot run. Resource failure is retained as an experimental outcome and never triggers an undocumented reduction in dataset size.

4 Persistence and inference workflow

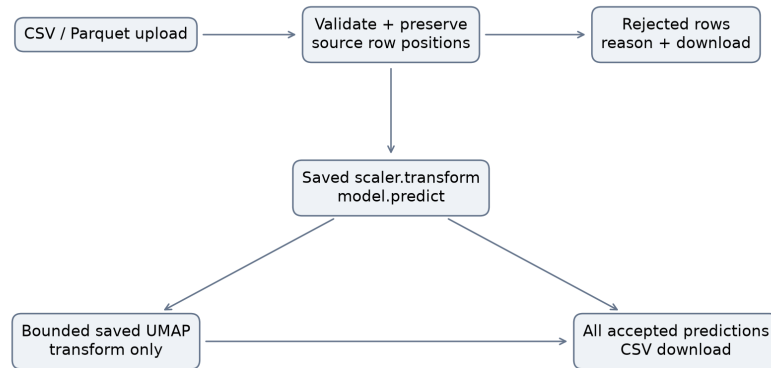


Figure 3. New-file prediction using immutable saved preprocessing and models.

4.3 Inference contract

The upload interface accepts CSV or Parquet, limits files to 20 MiB and source rows to 100,000, and validates the same schema. Every accepted row uses the saved scaler and `model.predict`. No new clusters are created during upload. Rejections are separately downloadable with their original positions.

A fixed-seed subset of at most 2,000 accepted upload rows passes through each saved UMAP reducer. Other rows retain cluster assignments but have blank projection coordinates. The chart therefore need not contain every exported prediction. Reference and uploaded points use different marker shapes.

4.4 Persistence and Colab fallback

JSON metadata records feature order, rules, population sizes and K; joblib stores trusted local estimators. A portable JSON snapshot records scaler means, variances and scales. The Colab notebook restores this state, verifies source hash and cleaned count, and invokes the same workers. It supports full ordinary estimators and an explicitly marked incremental MiniBatch path in one free CPU runtime.

No Colab execution is evidenced in the results. Workflow components and import validation are locally tested. Imported reports must identify environment and match the frozen experiment. Timing from another machine is not interpreted as a purely algorithmic difference.

5 Algorithm Implementation

5.1 Standard K-Means

Given standardized matrix Z and a chosen K , K-Means seeks centroids minimizing Equation (3). The implementation uses KMeans with $K=3$, $\text{random_state}=42$, $\text{n_init}=10$ and $\text{max_iter}=300$. Default initialization is k-means++, which favors separated starting centers; default optimization is Lloyd's method. Estimator defaults are version-dependent, so the installed version is recorded.

For each initialization, the assignment step selects the closest centroid for every trip. The update step replaces each centroid with the mean of its assigned vectors. These operations repeat until labels stabilize, the centroid-shift tolerance is met, or the iteration limit is reached. Among full K-Means initializations, the estimator retains the lowest-inertia solution. This is a local solution, not a proven global optimum.

$$c_i = \arg \min_c \|z_i - \mu_c\|^2, \quad \mu_c = \frac{1}{|C_c|} \sum_{i \in C_c} z_i \quad (5)$$

5.2 Implementation sequence

The shared function validates the bounded sample and selects the stored feature order. It fits one scaler, produces float32 standardized values and fits both models within a two-thread numerical limit. Training distances establish anomaly thresholds. Separate reducers are fitted on a shared reference subset, after which the bundle, metadata, profiles, centroids and sampled assignments are saved.

For n records, d features, K centers and I iterations, a conventional assignment-dominated cost is $O(nKdI)$ per initialization. Reading vectors and storing assignments also require memory. A memory-mapped source helps storage, but ordinary fitting still materializes working arrays. Memory feasibility is therefore evaluated independently of lazy preprocessing.

5.3 Strengths and limitations

The algorithm supplies a direct objective, interpretable centroids and simple inference. Limitations include scale and tail sensitivity, an imposed K , and preference for compact partitions. Related features such as distance, duration and speed affect the geometry jointly. The resulting groups are descriptions of a chosen representation rather than a uniquely determined taxonomy of travel behavior.

5 MiniBatch K-Means implementation

5.4 Mini-batch updates

MiniBatch K-Means updates centroids using small subsets rather than complete assignment/update cycles [3]. A conceptual update for an assigned observation uses accumulated count v_c and learning rate $1/v_c$ in Equation (6). Implementations combine such updates efficiently rather than requiring a Python loop per observation.

$$v_c \leftarrow v_c + 1, \quad \mu_c \leftarrow (1 - 1/v_c)\mu_c + (1/v_c)z_i \quad (6)$$

The application uses `batch_size=1000`, `K=3`, `seed 42`, `n_init=10` and `max_iter=300`. For MiniBatch, multiple initializations choose an initial configuration; `n_init` is not ten complete independent optimization runs. Convergence and center reassignment also follow the installed estimator's defaults [10].

5.5 Ordinary fitting and explicit streaming

Smaller comparisons call ordinary fit on the same matrix as K-Means. The dedicated full-data streaming experiment follows the same seeded ordering in blocks of up to 10,000 rows and calls `partial_fit` once per block. Its one pass contains 325 blocks. This distinct schedule is labelled streaming in all comparisons.

A batch update has approximately $O(bKd)$ assignment work for b rows, plus center updates. Total cost depends on initialization, update count and stopping behavior. A small batch reduces each update's cost but can increase variability. Larger batches may improve agreement while increasing update cost. Batches 100, 500, 1000, 5000 and 10000 are measured on identical 100,000-row data.

5.6 Interpretation of approximation

MiniBatch improves fit speed in these observations without guaranteeing the same partition. Numeric IDs are arbitrary and the final models' short-trip groups differ. Distortion, silhouette and ARI accompany runtime. Neither raw label equality nor a short fit timer establishes equivalent knowledge discovery or end-to-end ingestion performance.

5 Cluster selection evidence

5.7 Candidate evaluation

Originally, $K=2$ maximized sampled silhouette over $K=2..8$; it was measured, not hardcoded. The revised experiment evaluates $K=2..10$ on 20,000 hash-priority sampled accepted trips with one selection-specific scaler. Candidates use the same matrix and fixed 2,000-record silhouette subset. Stability is full-selection-sample ARI between seeds 42 and 43, each with ten initializations.

Table 3. Measured K-selection metrics on a fixed selection sample.

K	Inertia / row	Silhouette	Stability ARI	Min. share	Eligible
2	4.3575	0.4914	1.0000	12.670%	true
3	3.4586	0.3356	0.9999	11.585%	true
4	2.7490	0.3358	0.9998	0.205%	false
5	2.3238	0.3097	0.9896	0.200%	false
6	2.0771	0.3100	0.9982	0.200%	false
7	1.8867	0.3121	0.7350	0.025%	false
8	1.7097	0.2732	0.7220	0.025%	false
9	1.5706	0.2884	0.9346	0.025%	false
10	1.4681	0.2922	0.7282	0.025%	false

5.8 Declared procedure

Before the revised run, the procedure required minimum cluster share 0.5%, silhouette ≥ 0.25 and initialization ARI ≥ 0.80 . Among eligible candidates it maximizes normalized elbow strength $E(K)=1-x(K)-y(K)$, where x and y scale K and inertia between endpoint values. Ties favor fewer clusters. If no candidate qualifies, the code raises an error instead of choosing a default.

Only $K=2$ and $K=3$ qualify. $K=3$ improves inertia per trip from 4.357 to 3.459 and has the stronger eligible elbow. Its smallest cluster contains 2,317 trips (11.585%); stability is 0.999892. These are practical broad-pattern gates, not statistical significance tests.

5 Selection rationale and sensitivity

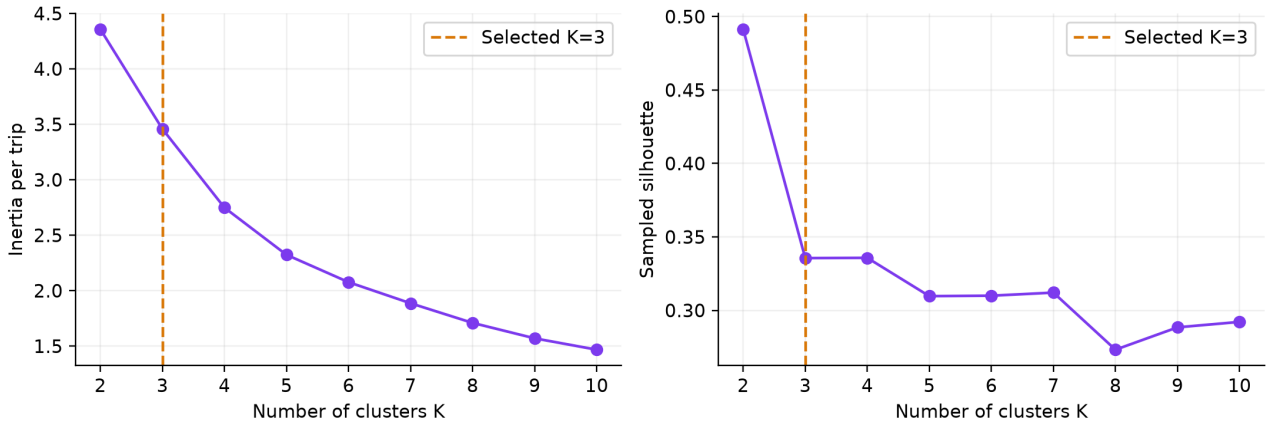


Figure 4. Elbow and silhouette analysis for K from 2 to 10 with K=3 marked.

5.9 Why K=2 was not retained

K=2 has silhouette 0.491, above K=3 at 0.336. However, the coarse split suppresses a stable timing distinction among short trips. At K=3, two groups average about two miles and twelve minutes with different circular pickup-hour centers. The third averages thirteen miles and thirty-six minutes. The decision combines this interpretation with inertia, shares and stability rather than treating one metric as decisive.

5.10 Why larger K was less suitable

K=4 creates a group of only 41 records (0.205%), averaging 0.012 miles and approximately \$1,861 per mile. Additional centroids may capture extreme charge intensity instead of broad patterns. K=7, 8 and 10 also fail the declared stability gate. These records remain in accepted data and anomaly analysis; they were not removed to improve visual separation.

Minimum shares from 0.25% through 1% retain K=3. Relaxing the minimum to 0.1% admits K=4,5,6 and favors K=5 under the same elbow score. K=3 is therefore a defensible broad-pattern resolution, not a unique natural cluster count. Two initializations test optimization stability, not stability across months or repeated independent population samples. The original K=2 artifacts and measurements are preserved separately.

6 Experimental Setup and Performance Evaluation

6.1 Recorded hardware and software

The target is Windows 10 with 8 GB installed RAM. The measured runtime identifies Windows 11 build 10.0.26200, eight logical CPUs and 5.889 GiB usable RAM. CPU model and physical installed capacity were not recorded; they are not inferred from logical CPU count. Available RAM during the successful sweep was approximately 1.0–1.36 GiB.

Table 4. Recorded runtime and estimator configuration.

Setting	Recorded value
Operating system	Windows 11, build 26200
Memory / logical CPUs	5.889 GiB / 8; CPU model not recorded
Python / numerical threads	3.12.13 / 2
NumPy / Polars	2.5.3 / 1.44.2
scikit-learn / UMAP	1.9.1 / 0.5.12
K / seed / features	3 / 42 / 6 standardized float32 columns
Initialization / n_init	k-means++ / 10
KMeans / MiniBatch max_iter	300 / 300
Main MiniBatch size	1,000
Silhouette evaluation	2,000 reproducibly selected records

6.2 Controlled design

A seeded permutation of accepted rows gives nested 50K, 100K, 250K, 500K and 1M prefixes. Both estimators use identical data, feature order, scaler, K and seed. Saved demonstration-model fits are separate. Inertia and cluster sizes are recomputed over every benchmark row in bounded chunks. Silhouette and ARI use a common fixed 2,000-row subset within each size.

The fit timer excludes ingestion, scaling, initial matrix selection and metrics. Streaming includes block extraction within its timer. Peak memory is process-tree RSS sampled every 50 ms across startup, fit and evaluation, not estimator-only allocation. Each setting has one repetition; no confidence intervals are available.

6.3 Resource controls

Workers stop below 512 MiB available RAM or at 900 seconds. Ordinary fits are skipped when estimated working memory exceeds 60% of available RAM. Failed or skipped runs retain their reason, and a fingerprint prevents silent mixing of incompatible frozen configurations.

6 Runtime and memory results

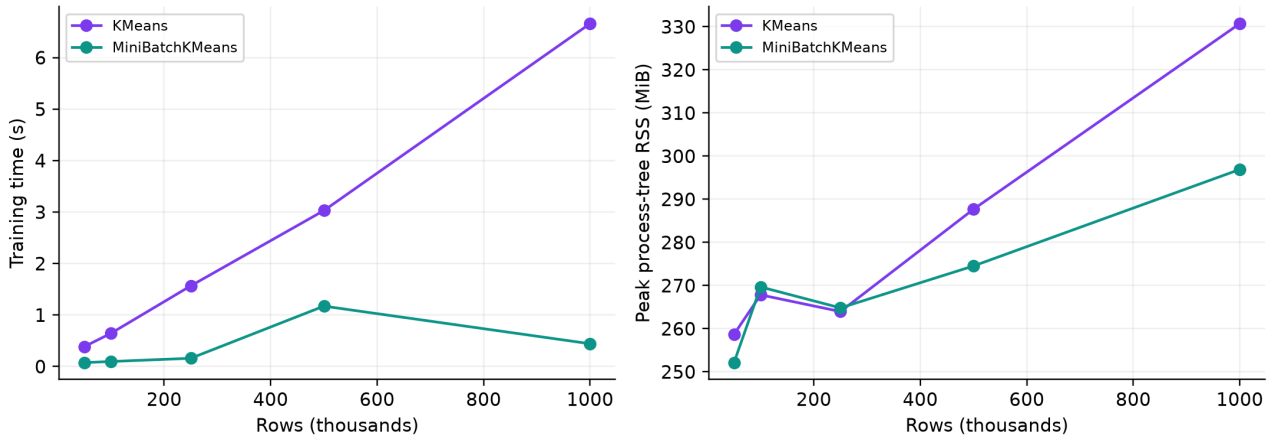


Figure 5. Fitting time and process-tree RSS at the five paired dataset sizes.

Table 5. Paired fit times and peak RSS for K=3.

Rows	Algorithm	Fit seconds	Peak RSS MiB
50K	KM	0.3866	258.69
50K	MB	0.0762	252.14
100K	KM	0.6449	267.85
100K	MB	0.0978	269.68
250K	KM	1.5677	263.98
250K	MB	0.1604	264.83
500K	KM	3.0292	287.71
500K	MB	1.1732	274.55
1000K	KM	6.6608	330.73
1000K	MB	0.4426	296.92

K-Means grows from 0.387 seconds at 50K to 6.661 seconds at 1M. MiniBatch takes 0.076 and 0.443 seconds at these endpoints, an observed fit-time ratio of 15.05 at 1M. This is a single-machine result excluding preparation and evaluation.

MiniBatch time is not monotonic: the 500K observation is slower than 1M. Early stopping, scheduling and memory load can affect a single measurement. Graph lines connect actual observations and are not interpolated predictions. Peak RSS contains substantial process overhead; lower measured RSS at a larger size does not imply decreasing asymptotic requirements.

6 Inertia silhouette and agreement

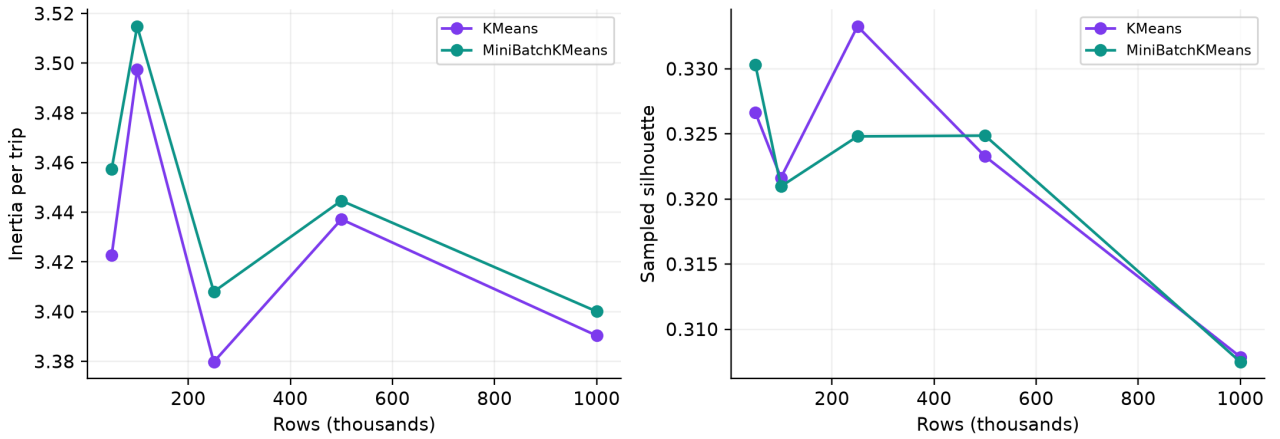


Figure 6. Normalized inertia and sampled silhouette for identical paired inputs.

Table 6. Quality metrics and sampled partition agreement.

Rows	Algorithm	Inertia / row	Silhouette	ARI vs KM
50K	KM	3.4227	0.3266	Reference
50K	MB	3.4574	0.3303	0.8090
100K	KM	3.4974	0.3216	Reference
100K	MB	3.5149	0.3210	0.9537
250K	KM	3.3798	0.3333	Reference
250K	MB	3.4080	0.3248	0.8170
500K	KM	3.4372	0.3233	Reference
500K	MB	3.4447	0.3249	0.7970
1000K	KM	3.3903	0.3079	Reference
1000K	MB	3.4001	0.3075	0.9873

At 1M, MiniBatch inertia is 3.4001 versus 3.3903, approximately 0.287% higher. Silhouettes are 0.3075 and 0.3079; ARI is 0.9873. This indicates strong agreement for that evaluation subset, not identical final saved 100K partitions.

Agreement varies from about 0.797 at 500K to 0.987 at 1M. A slightly higher silhouette for an approximate solution at some sizes does not imply better minimization of the K-Means objective: the metrics describe different properties. No ground-truth travel classes exist, so neither metric is classification accuracy.

6 Full-data and batch-size experiments

6.4 Complete accepted population

Both MiniBatch paths processed 3,241,580 records. Ordinary fit took 1.4641 seconds, peak RSS 422.54 MiB, inertia/trip 3.3922 and silhouette 0.3303. One-pass streaming took 1.2232 seconds, peak RSS 284.32 MiB, inertia/trip 3.3965 and silhouette 0.3348. Schedules and timing boundaries differ, limiting direct causal interpretation of their runtime difference.

Full ordinary K-Means was skipped: estimated 997.94 MiB exceeded 60% of 1,111.70 MiB available. No full K-Means fit time or paired full-size ARI exists. Colab remains an unexecuted fallback.

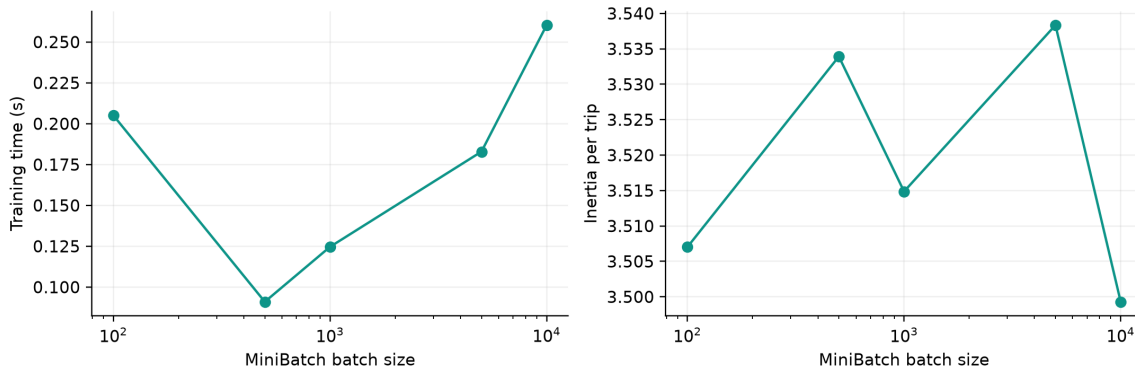


Figure 7. Batch-size performance on one fixed 100000-record prefix.

Table 7. Completed MiniBatch batch-size study.

Batch size	Fit seconds	Inertia / row	Silhouette	ARI vs KM
100	0.2051	3.5070	0.3173	0.7665
500	0.0910	3.5339	0.3192	0.8135
1000	0.1247	3.5149	0.3210	0.9537
5000	0.1829	3.5383	0.3191	0.8045
10000	0.2603	3.4993	0.3218	0.9891

Batch 500 was fastest in this run; batch 10000 gave the lowest inertia and highest silhouette and agreement among tested settings. This is a trade-off, not a universal optimum. The application default remains 1000; the study did not retrospectively alter the finalized configuration.

7 Cluster Analysis and Knowledge Discovery

7.1 Populations and descriptive labels

Profiles summarize all 100,000 records used to fit each saved model, not all accepted trips or benchmark assignments. Names express measured means and timing patterns. No destination, geographic zone or passenger purpose is inferred.

Table 8. Saved-model cluster counts and mean trip characteristics.

Model / ID	Trips	Share %	Miles	Minutes	mph	USD/mi
KM / 0	48,361	48.36	2.00	11.85	10.16	16.35
KM / 1	40,273	40.27	1.83	12.32	9.19	8.95
KM / 2	11,366	11.37	13.08	35.83	23.95	4.23
MB / 0	32,527	32.53	2.17	11.38	11.48	7.75
MB / 1	55,937	55.94	1.76	12.41	8.68	16.06
MB / 2	11,536	11.54	12.99	35.72	23.82	4.23

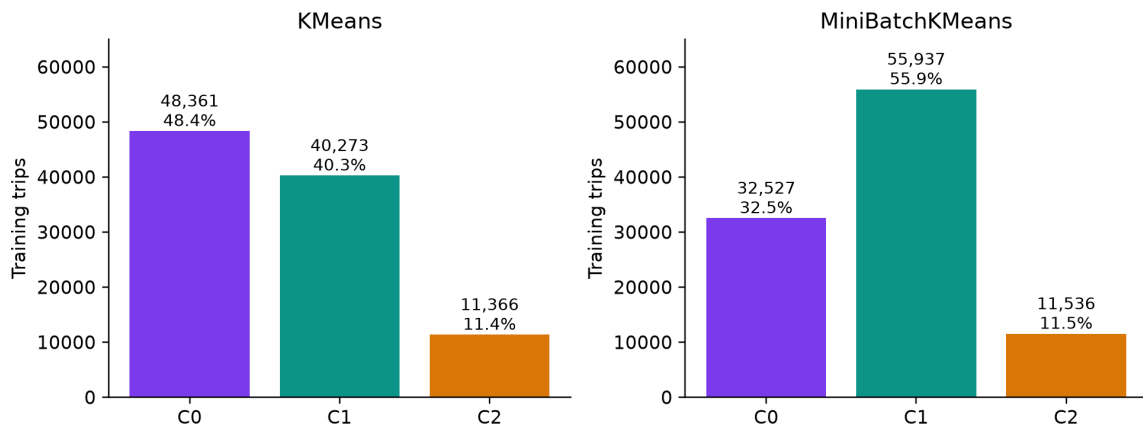


Figure 8. Training cluster distributions for both saved algorithms.

K-Means C0 and C1 are shorter trips with distinct timing profiles; C2 is longer and faster on average. MiniBatch C2 is similarly longer in this fit, but equal IDs do not guarantee equivalence. The short-trip groups differ in sizes and timing. Categorical colors remain fixed within each selected model, not an alignment between models.

7 Interpretation of K-Means profiles

7.2 C0 Short trips centered on evening pickup

C0 contains 48,361 trips (48.361%). Mean distance is 2.003 miles, duration 11.855 minutes, speed 10.162 mph and fare per mile \$16.351. Its circular pickup-hour center is about 19.73 hours, displayed around 20:00. This aggregate center does not mean every trip occurs then. High fare intensity is sensitive to very short distances.

7.3 C1 Short trips centered on late morning

C1 contains 40,273 trips (40.273%). Mean distance is 1.827 miles, duration 12.320 minutes, speed 9.188 mph and fare per mile \$8.953. Its circular hour center is about 11.28 hours. C0 and C1 demonstrate why cyclic timing matters: trip scale alone obscures their selected-feature distinction.

7.4 C2 Longer faster trips with mixed timing

C2 contains 11,366 trips (11.366%), averaging 13.081 miles, 35.828 minutes, 23.953 mph and \$4.229 per mile. Its circular concentration is below the dashboard's 0.4 naming threshold, so it has mixed pickup hours. Airport travel, highway use or specific destinations are not established.

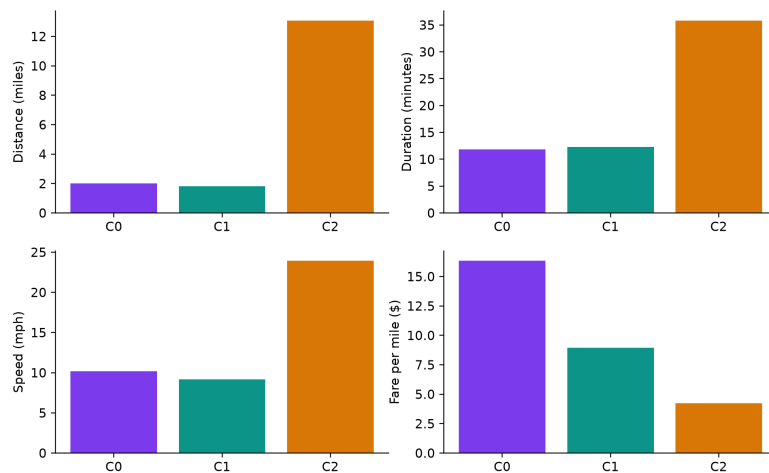


Figure 9. Mean distance duration speed and fare intensity for K-Means groups.

7 UMAP projections and interpretation limits

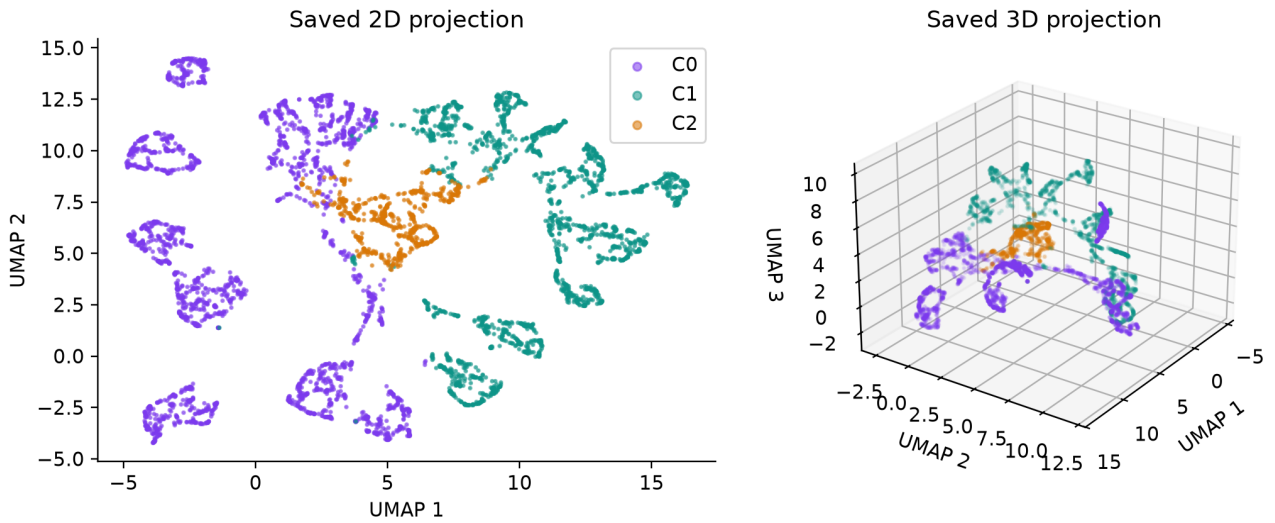


Figure 10. Saved 2D and 3D UMAP coordinates for the same 5000 reference trips.

7.5 Projection method

Two separately fitted reducers use the same bounded reference sample and standardized features. Their dimensions are two and three, with fifteen neighbors, fixed random and transform seeds and one UMAP job. This figure is generated from saved coordinates, not a refit or a screenshot of a new experiment.

The dashboard supports 3D rotation and zoom, hover details and cluster filtering. New-file points use transform and are highlighted against the reference. Sampling limits browser memory and rendering cost.

7.6 Limits of the visual evidence

UMAP axes have no physical units and are not geographic coordinates. Nonlinear projection can distort distances and gaps. Separated islands do not independently establish $K=3$. Silhouette, inertia and centroid distances are evaluated in standardized feature space.

Only 5,000 training records are shown. The view illustrates patterns rather than enumerating the training population. The same geometry can be colored by either model, allowing partition differences to be examined without changing the coordinate system. Transformation of future data also assumes sufficient compatibility with the distribution learned by the saved reducer [8].

8 Anomaly Detection

8.1 Distance scores and percentile threshold

For each accepted trip, r_i is its Euclidean distance to the predicted centroid in standardized space. The model threshold q is the 99th percentile of training distances. A trip is flagged if $r_i > q$; severity is r_i/q . One marks the boundary, not a risk probability or an independently calibrated hypothesis test.

K-Means uses $q=4.023440$ and MiniBatch $q=4.032615$. Each flags 1,000 of 100,000 training records (1%). This follows the chosen training percentile and is not estimated fraud prevalence. An uploaded file may have a different rate.

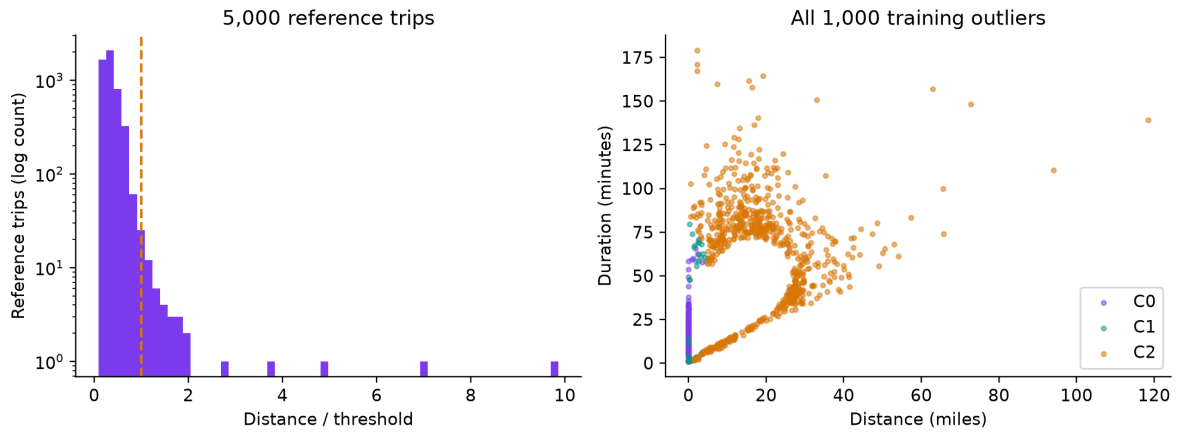


Figure 11. Reference-score distribution and all K-Means training outliers.

Table 9. Training outlier counts by model-specific cluster.

Model	Cluster 0	Cluster 1	Cluster 2	Total / training
KMeans	233	34	733	1,000 / 100,000
MiniBatch	18	248	734	1,000 / 100,000

The largest K-Means ratio is 23.900: a recorded 0.01-mile trip has fare \$95.92 and duration 53.63 minutes, yielding \$9,592 per mile. It warrants possible source review, but its cause is unknown. Of K-Means flags, 733 belong to the longer-trip C2 group. A global threshold flags unequal proportions across groups; cluster-conditional thresholds and independent anomaly methods are future work.

9 Application Implementation

9.1 Overview and Cluster Discovery

Overview prioritizes source, accepted and training counts, frozen K, active model and anomaly status. Detailed metadata is expandable. Cluster Discovery supplies a cluster selector, 2D/3D tabs, distribution, feature comparisons and inverse-scaled centroids. The following images are actual saved dashboard captures.

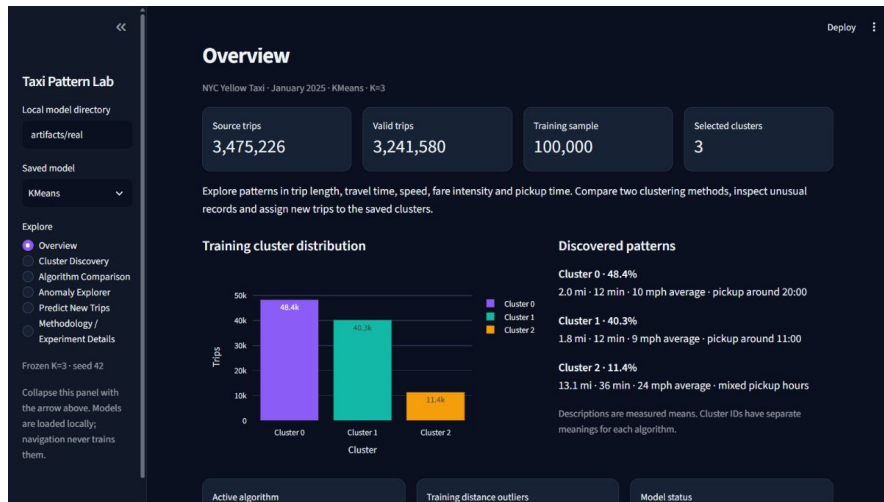


Figure 12. Overview page with measured populations and categorical profiles.

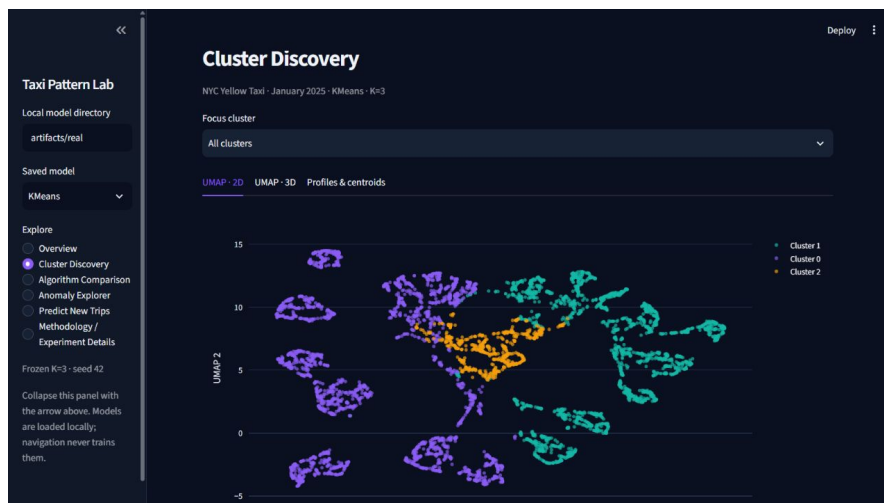


Figure 13. Cluster Discovery page displaying the saved reference projection.

The sidebar collapses to free chart space. Colors identify categories rather than a continuous numerical magnitude. Selecting an algorithm changes the viewed assignments; navigation loads saved artifacts and never trains an estimator.

9 Comparison and anomaly pages

9.2 Algorithm Comparison and Anomaly Explorer

Comparison presents successful measurements and explicit skipped or failed statuses, separating environment and streaming mode. Additional tabs show batch sizes and K-selection evidence. Anomaly Explorer supplies cluster/severity filters, a bounded histogram, unusual-trip scatter and matching-record downloads.

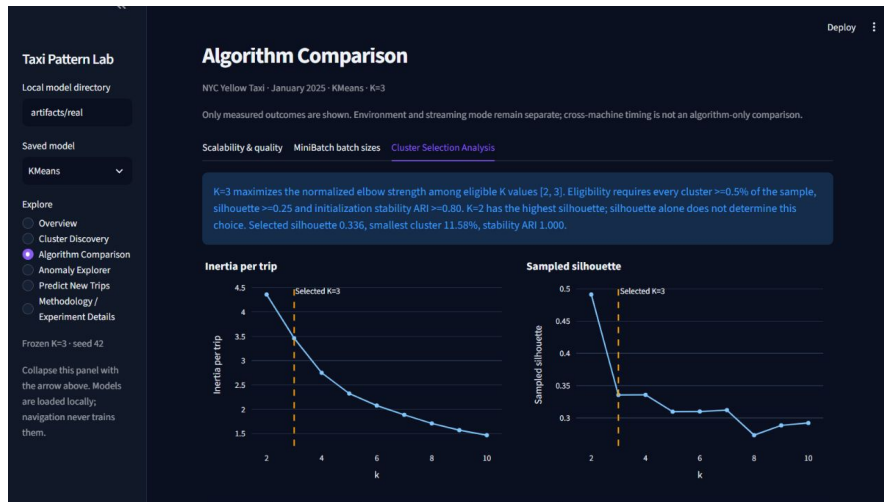


Figure 14. Actual Algorithm Comparison capture showing cluster-selection analysis.

SCREENSHOT PLACEHOLDER

Anomaly Explorer

A verified page capture was unavailable for this report.

This panel is not a reconstructed application screenshot.

Figure 15. Anomaly Explorer screenshot placeholder; capture unavailable.

Missing measurements are not plotted as zero. Imported Colab reports must pass schema and configuration checks. Anomaly language describes unusual observations rather than confirmed fraud, and the displayed population is the model-training sample. Plot data and numerical outputs are independently available in the preceding chapters even if a page capture is unavailable.

9 Prediction and methodology pages

9.3 Predict New Trips and experiment details

Prediction validates files and reports accepted/rejected counts. The saved estimator supplies labels, descriptive names, distances and severity ratios. Methodology explains features, configurations, runtime metadata and experimental limitations.



Figure 16. Predict New Trips screenshot placeholder; capture unavailable.



Figure 17. Methodology screenshot placeholder; capture unavailable.

The real demonstration used 30 hash-ranked cleaned records immediately after the 100K training prefix. All 30 passed CSV and Parquet checks for both algorithms, with stable labels and finite 2D/3D coordinates. A browser CSV upload and downloaded output were verified. This establishes inference consistency, not accuracy against known classes. Actual output rows are included in Appendix B.

10 Results and Discussion

10.1 Consolidated findings

The project converts a 3.475-million-record source into a traceable analytical population with exclusive rejection counts. This matters because algorithm scaling cannot be compared fairly if different cleaning decisions silently change the input. The source hash and frozen snapshot allow the accepted population and feature geometry to be reviewed together.

MiniBatch reduces measured fitting time at all five paired sizes, with slightly higher normalized inertia. At 1M, the fit-time ratio is about 15.05, inertia differs by 0.287% and ARI is 0.9873. The smallest paired ARI is about 0.797, demonstrating that an approximation can change assignments while aggregate quality metrics remain close.

10.2 Memory and practical usefulness

Full ordinary MiniBatch and one-pass streaming process the monthly accepted population; preflight prevents full ordinary K-Means under available memory. This supports incremental processing under the tested conditions. It does not prove that conventional K-Means cannot fit on an 8 GB machine with another background load or implementation.

Saved 100K models enable demonstration without reloading millions of trips. Cached bundles and predictions avoid repeated work, while bounded samples constrain chart cost. Initial UMAP import or compilation can delay the first interaction. Browser automation delays were not interpreted as estimator runtime measurements.

10.3 Threats to validity

There is one month, one final representation and one timing repetition per setting. Selection and training samples overlap; no ground-truth cluster labels exist. Initialization stability uses two seeds. Scaling does not remove the fare-intensity tail or feature correlation. These limitations restrict generalization.

Anomaly counts follow the selected threshold, not discovery of a verified class. Holdout checks establish identity, persistence and transformation compatibility rather than correctness of semantic names. Independent samples, repeated timings and feature-transform sensitivity should precede stronger claims about stable travel categories or deployment-wide performance.

11 Conclusion and Future Scope

11.1 Conclusion

Taxi Pattern Lab implements official data ingestion, validated feature engineering, justified K-selection, paired clustering models, reproducible experiments, interpretation and local inference. K=3 is supported as a broad-pattern resolution through declared quality gates and inertia-elbow strength. The higher silhouette of K=2 and sensitivity to the size gate remain explicitly documented.

Both algorithms complete 50K–1M comparisons. MiniBatch additionally processes the full accepted dataset through ordinary and incremental paths. The full K-Means preflight skip preserves research integrity instead of substituting a smaller fit. Twenty-five offline tests and real holdout checks support the prediction path, while the six-page application exposes evidence for evaluation.

The central finding is a measured speed–quality trade-off. MiniBatch is faster in these runs, with slightly higher distortion and agreement that varies across sizes. The report consequently retains several metrics and resource outcomes rather than declaring one algorithm universally best.

11.2 Future scope

Multi-month data could assess seasonal change and profile persistence. Independent samples and repeated timings could quantify uncertainty. Log or robust scaling of charge intensity could test whether rare records unduly influence the objective.

Gaussian mixtures, density-based clustering and hierarchical summaries could examine alternative geometry. Geographic analysis would require coordinates or meaningful zone encodings rather than Euclidean use of integer IDs. Cluster-conditional anomaly thresholds and reviewed examples could improve interpretation of unusual records.

Incremental refresh, drift monitoring and versioned deployment could support ongoing arrivals. Same-runtime cloud experiments could complete full conventional K-Means when resources permit. These are future extensions: Colab is an unexecuted fallback, and current uploads do not update learned parameters.

References

- [1] NYC Taxi and Limousine Commission, “TLC Trip Record Data,” January 2025 Yellow Taxi Parquet. Available: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>. Accessed: Oct. 8, 2026.
- [2] S. P. Lloyd, “Least squares quantization in PCM,” IEEE Transactions on Information Theory, vol. 28, no. 2, pp. 129–137, 1982, doi: 10.1109/TIT.1982.1056489.
- [3] D. Sculley, “Web-scale k-means clustering,” Proceedings of the 19th International Conference on World Wide Web, pp. 1177–1178, 2010, doi: 10.1145/1772690.1772862.
- [4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. Available: <https://www.jmlr.org/papers/v12/pedregosa11a.html>.
- [5] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” Journal of Computational and Applied Mathematics, vol. 20, pp. 53–65, 1987, doi: 10.1016/0377-0427(87)90125-7.
- [6] Scikit-learn developers, “adjusted_rand_score.” Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.adjusted_rand_score.html. Accessed: Oct. 8, 2026.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” arXiv:1802.03426, 2018. Available: <https://arxiv.org/abs/1802.03426>.
- [8] UMAP developers, “Transforming New Data with UMAP.” Available: <https://umap-learn.readthedocs.io/en/latest/transform.html>. Accessed: Oct. 8, 2026.
- [9] Streamlit, “Streamlit documentation.” Available: <https://docs.streamlit.io/>. Accessed: Oct. 8, 2026.
- [10] Scikit-learn developers, “MiniBatchKMeans.” Available: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.MiniBatchKMeans.html>. Accessed: Oct. 8, 2026.

Appendix A Installation and repository guide

A.1 Repository and source snapshot

Repository: https://github.com/vighneshvijayjadhav-eng/large-scale_clustering_comparison

The application delivery was verified at a381277dd8dcadc13cc16c85f24fcdcf1850b2e2. The benchmark manifest records preceding HEAD 341b5da because K=3 changes were still uncommitted during measurement. Delivered source includes those changes; historical metadata is not rewritten.

Table 10. Repository components and reproducibility roles.

Path	Purpose
src/taxi/	Validation, fitting, inference, benchmark and plotting
app.py; .streamlit/	Local dashboard and theme
config/; tests/	Reproducible settings and offline tests
reports/	Measured CSV/JSON results and this report
scripts/; notebooks/	Smoke checks, report generation and Colab fallback
data/; artifacts/	Ignored local data and saved model binaries

A.2 Windows PowerShell commands

Install Python 3.11 or 3.12 and execute from the repository root. Application dependencies install together. Data and generated models remain outside Git.

```
py -3.12 -m venv .venv
.\venv\Scripts\python -m pip install -e ".[dev]"
.\venv\Scripts\taxi download
.\venv\Scripts\taxi clean
.\venv\Scripts\taxi train data/clean.parquet --rows 100000
.\venv\Scripts\taxi benchmark --full
.\venv\Scripts\streamlit run app.py
```

For an offline synthetic demo, run taxi demo and select artifacts/demo. Fixtures are development-only. The free Colab notebook is notebooks/full_dataset_benchmark_colab.ipynb; run its cells in order and download CSV/JSON results. Import compatible results through Algorithm Comparison or reports/colab. Do not mix different scalers, K values or features.

Appendix B Inputs outputs and validation

B.1 Real unseen-record examples

The following rows come from saved real K-Means holdout output, not a synthetic illustration. `row_id` preserves input position. Every accepted row is exported, even if chart sampling leaves projection coordinates blank.

Table 11. Actual examples from the disjoint 30-record holdout.

row_id	Miles	Minutes	Hour	Cluster	Anomaly ratio
0	2.39	14.917	20	0	0.1215
1	1.09	4.133	9	1	0.3829
2	2.57	19.583	18	0	0.2592

Required fields are `tped_pickup_datetime`, `tped_dropoff_datetime`, `trip_distance`, `fare_amount` and `total_amount`. CSV timestamps allow space or T separators; Parquet datetime columns are supported. Outputs add features, `cluster`, `centroid_distance`, `anomaly_ratio`, `distance_outlier` and bounded UMAP2/3 coordinates. The dashboard also adds `cluster_description`.

B.2 Testing summary

During report preparation, 25 offline tests passed. Coverage includes invalid timestamps and numbers, zero-division paths, feature order, persistence, row identity, CSV/Parquet upload, selection outputs, profiles, colors, empty inputs, missing artifacts, all six pages and report-import validation. Real smoke checks confirm both algorithms and saved 2D/3D transforms for 30 disjoint records. CI needs no full-data download.

```
.\.venv\Scripts\python -m pytest -q
.\.venv\Scripts\ruff check .
.\.venv\Scripts\ruff format --check .
.\.venv\Scripts\python scripts/smoke_saved.py
```

B.3 Reproducibility checklist

Keep `config/default.json`, source hash, cleaned count, frozen scaler, $K=3$, seed 42, versions and experiment fingerprint together. Regenerate dependent artifacts after configuration changes. Timing varies with load; unsuccessful runs retain status. GitHub Actions targets Windows Python 3.11/3.12, but remote execution was not independently verified. Report preparation performs no real-model retraining.